

INFORMACIÓN TÉCNICA PROYECTO CENTRO DEPORTIVO

ÍNDICE

1. Introducción
2. Arquitectura MVVM
3. Arquitectura por Capas
4. Estructura de Proyectos
5. Descripción Detallada de Componentes
 - 5.1. Modelo de Datos
 - 5.2. ViewModels
 - 5.3. Views
 - 5.4. Commands
 - 5.5. Acceso a Datos
 - 5.6. Informes
6. Modelo de Base de Datos
7. Flujo de Trabajo de la Aplicación

1. INTRODUCCIÓN

El proyecto **Centro Deportivo** es una aplicación de escritorio desarrollada con **WPF** sobre **.NET Framework 4.8**.

Su finalidad es gestionar las operaciones de un centro deportivo: gestión de socios, actividades deportivas y reservas con control de aforo y visualización de informes.

La aplicación sigue el patrón **MVVM**, que permite una separación clara entre la interfaz de usuario y la lógica de la aplicación. Para el acceso a datos se utiliza **Entity Framework 6** como ORM, trabajando contra una base de datos **SQL Server**. La generación de informes se realiza mediante **Crystal Reports**.

2. ARQUITECTURA MVVM

El patrón **MVVM** se utiliza para separar responsabilidades dentro de la aplicación y facilitar el mantenimiento y la escalabilidad del código.

En esta arquitectura, el **Model** representa los datos y el acceso a la base de datos, la **View** define la interfaz gráfica y el **ViewModel** actúa como intermediario entre ambos, conteniendo la lógica de presentación.

El intercambio de información entre la View y el ViewModel se realiza mediante **DataBinding**, mientras que las acciones del usuario se gestionan a través de **Commands**, evitando el uso de lógica en el code-behind de las vistas.

Componentes principales del patrón MVVM

- **Model**
Representa los datos de la aplicación. Incluye las entidades generadas automáticamente por Entity Framework a partir de la base de datos y no tiene dependencia directa con las vistas ni con los ViewModels.
- **View**
Define la interfaz gráfica mediante XAML. Se enlaza al ViewModel mediante DataBinding y no contiene lógica de negocio.
- **ViewModel**
Contiene la lógica de presentación. Expone propiedades y comandos que la View consume. Implementa la interfaz INotifyPropertyChanged para notificar cambios a la interfaz.
- **Commands**
Encapsulan las acciones del usuario, como pulsaciones de botones, conectando la interfaz con métodos del ViewModel de forma desacoplada.

3. ARQUITECTURA POR CAPAS

La aplicación **Centro Deportivo** está diseñada siguiendo una **arquitectura por capas**, lo que permite una separación de responsabilidades y un mejor control de las dependencias entre los componentes del sistema.

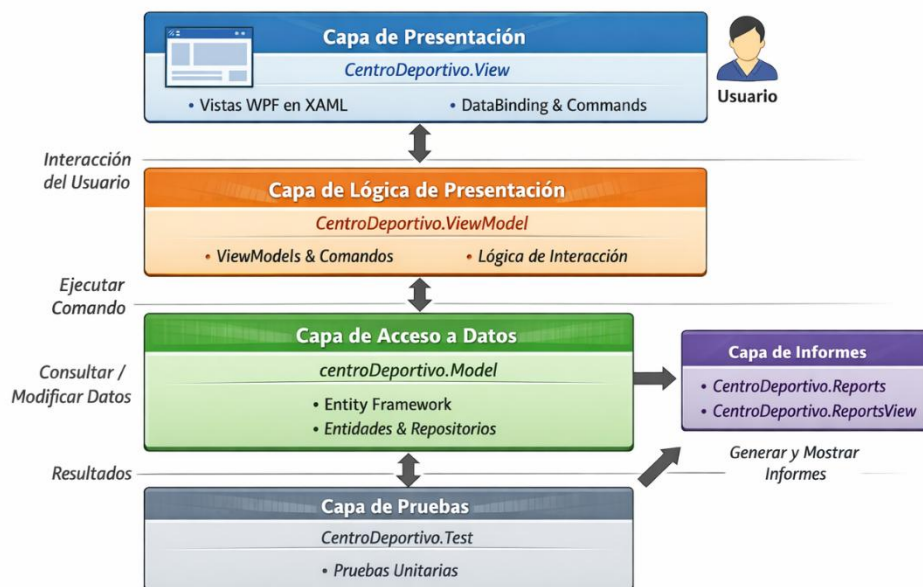
La **capa de presentación** corresponde al proyecto *CentroDeportivo.View* y contiene las ventanas WPF definidas en XAML. Su función es mostrar la información al usuario y capturar sus acciones, delegando toda la lógica de funcionamiento en los ViewModels mediante mecanismos de *DataBinding* y *Commands*.

La **capa de lógica de presentación**, implementada en el proyecto *CentroDeportivo.ViewModel*, agrupa los ViewModels y los comandos de la aplicación. Esta capa actúa como intermediaria entre la interfaz y los datos, gestionando la interacción del usuario, validando la información introducida y coordinando el acceso a la capa de datos.

La **capa de acceso a datos** se encuentra en el proyecto *centroDeportivo.Model* y contiene las entidades generadas por Entity Framework, el contexto de base de datos y los repositorios. Su responsabilidad es encapsular el acceso a la base de datos y evitar que el resto de capas dependan directamente del ORM.

La **capa de informes**, formada por los proyectos *CentroDeportivo.Reports* y *CentroDeportivo.ReportsView*, se encarga de la definición, construcción y visualización de los informes generados mediante Crystal Reports.

Finalmente, la **capa de pruebas**, incluida en el proyecto *CentroDeportivo.Test*, contiene pruebas unitarias destinadas a validar el correcto funcionamiento de los ViewModels, las validaciones y los repositorios.



4. ESTRUCTURA DE PROYECTOS

La solución está compuesta por seis proyectos principales, cada uno con una responsabilidad bien definida:

- **centroDeportivo.Model:** Biblioteca de clases que contiene las entidades generadas por Entity Framework, el contexto de base de datos y los repositorios de acceso a datos.
- **CentroDeportivo.ViewModel:** Biblioteca de clases que contiene los ViewModels, comandos y validaciones de la aplicación.

- **CentroDeportivo.View**: Aplicación WPF que implementa la interfaz gráfica de usuario mediante ventanas XAML.
- **CentroDeportivo.Reports**: Biblioteca de clases encargada de la definición y construcción de los informes mediante Crystal Reports y DataSets tipados.
- **CentroDeportivo.ReportsView**: Biblioteca WPF que implementa el visor de informes utilizando CrystalReportViewer.
- **CentroDeportivo.Test**: Proyecto de test para validar el comportamiento de la aplicación.

5. DESCRIPCIÓN DETALLADA DE COMPONENTES

5.1. MODELO DE DATOS

El modelo de datos representa la información gestionada por la aplicación y está generado automáticamente mediante Entity Framework en modo **Database First**.

Las entidades principales son **Socios**, **Actividades** y **Reservas**, que reflejan directamente las tablas de la base de datos y mantienen relaciones de uno a muchos entre ellas.

El acceso a estas entidades se realiza a través del patrón **Repository**, que encapsula la lógica de acceso a datos y evita dependencias directas con el ORM.

5.2. VIEWMODELS

Los ViewModels contienen la lógica de presentación y actúan como intermediarios entre las vistas y el modelo de datos.

Todos los ViewModels heredan de una clase base que implementa `INotifyPropertyChanged`, permitiendo la actualización automática de la interfaz cuando cambian las propiedades.

Entre los ViewModels principales se encuentran:

- **MenuPrincipalViewModel**, encargado de mostrar estadísticas generales del centro.
- **SociosViewModel**, que gestiona el alta, edición y eliminación de socios.
- **ActividadesViewModel**, responsable de la gestión de actividades deportivas.

- **ReservasViewModel**, que gestiona la creación y validación de reservas, incluyendo la comprobación de aforo disponible.

5.3. VIEWS

Las Views son las ventanas WPF definidas en XAML que conforman la interfaz gráfica de la aplicación.

Cada ventana tiene asignado un ViewModel como DataContext y utiliza DataBinding para enlazar controles visuales con propiedades y comandos. La lógica en el code-behind se reduce al mínimo indispensable.

Las vistas principales incluyen el menú principal, la gestión de socios, actividades y reservas.

5.4. COMMANDS

Los Commands permiten enlazar acciones del usuario con métodos del ViewModel sin acoplar la lógica a la interfaz.

La aplicación utiliza una implementación genérica de ICommand mediante la clase **RelayCommand**, que recibe un método de ejecución y, opcionalmente, una condición que indica si el comando puede ejecutarse.

El uso de Commands permite que los controles se habiliten o deshabiliten automáticamente según el estado de la aplicación.

5.5. ACCESO A DATOS

El acceso a datos se implementa en forma de repositorios, que centraliza las operaciones CRUD y encapsula el uso de Entity Framework.

5.6. INFORMES

El sistema de informes utiliza **Crystal Reports** para generar documentos PDF a partir de los datos de la aplicación.

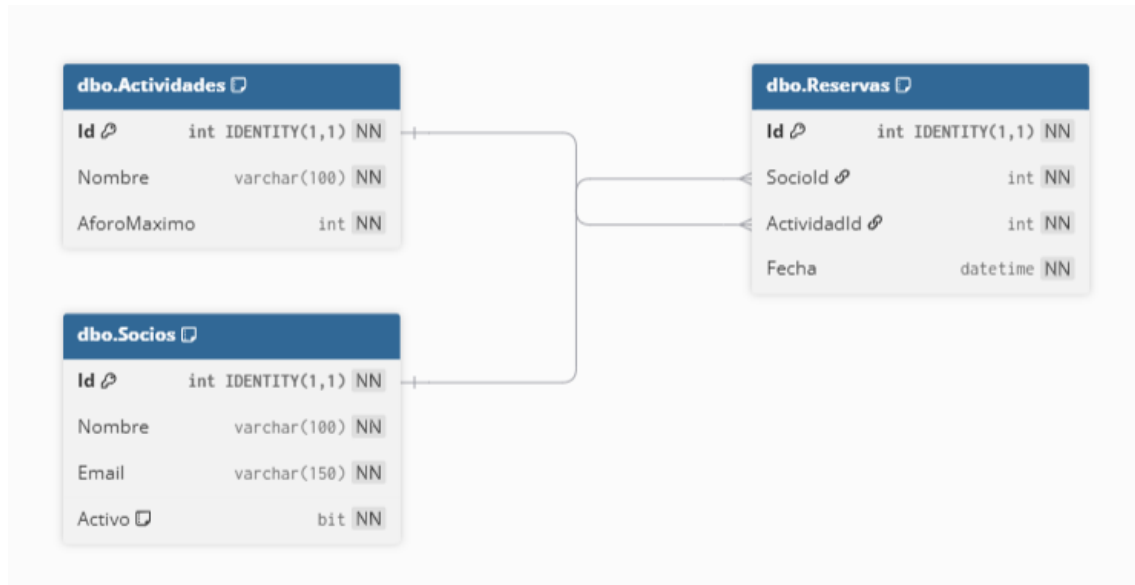
La generación de informes sigue un flujo definido: la vista solicita un informe, se selecciona el tipo correspondiente, se construyen los datos mediante un ReportBuilder, se cargan en un DataSet tipado y finalmente se visualizan mediante un visor de informes.

Los informes disponibles incluyen el listado maestro de socios, las reservas por actividad y el historial completo de reservas.

6. MODELO DE BASE DE DATOS

La base de datos **CentroDeportivo** está implementada en SQL Server y se compone de las tablas **Socios**, **Actividades** y **Reservas**.

Las relaciones entre tablas son de uno a muchos: un socio puede tener varias reservas y una actividad puede estar asociada a múltiples reservas. Estas relaciones se implementan mediante claves foráneas.



7. FLUJO DE TRABAJO DE LA APLICACIÓN

El flujo típico de la aplicación comienza con la carga del menú principal y la visualización de estadísticas generales. Desde este menú, el usuario puede acceder a la gestión de socios, actividades o reservas.

Cada acción del usuario se procesa a través de los ViewModels, que validan los datos, acceden a los repositorios y actualizan la interfaz de forma automática. La generación de informes se integra en el flujo normal de uso, permitiendo obtener documentos en cualquier momento.

INFORMACIÓN FINAL

Proyecto: Centro Deportivo – Aplicación WPF

Tecnologías: C#, WPF, MVVM, Entity Framework 6, SQL Server, Crystal Reports

Framework: .NET Framework 4.8

Fecha: 2026